AI Security Findings

Overall risk: CRITICAL | Total findings: 19

Executive summary

Risk overview

Total findings: 19. Critical: 5, High: 9, Medium: 5, Low: 0.

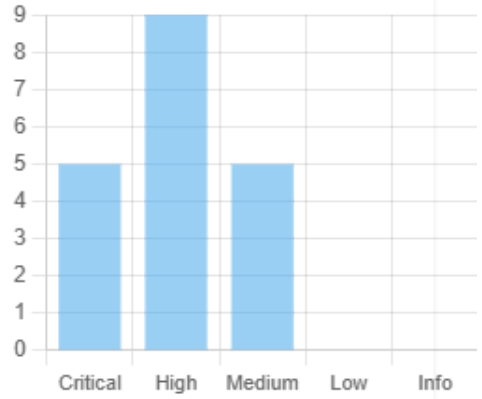
Technical summary

The overall risk level for this project is critical due to multiple severe vulnerabilities that could lead to catastrophic compromise. Top priority issues include critical command injection vulnerabilities where untrusted input is used directly in shell commands and Python's exec() function, allowing arbitrary code execution. Additionally, the autonomous agent operates with broad, unrestricted privileges capable of destructive actions without human approval, posing a major abuse risk. Hardcoded secrets such as API keys, database credentials, and internal system prompts containing sensitive information exacerbate the risk of credential leakage and insider threats. The system also lacks proper validation and sanitization in several areas, enabling SQL injection, stored XSS, and data poisoning attacks. Furthermore, high-stakes decisions driven solely by LLM output without human oversight risk critical business and ethical errors.

Immediate remediation should focus on eliminating command injections by avoiding shell=True, adopting parameterized queries, sandboxing or validating LLM-generated code, and enforcing strict scope and human-in-the-loop controls for autonomous agent actions. Secrets must be removed from source code and managed securely using environment variables or secrets management. Output escaping and input validation are crucial to mitigate XSS and injection risks. Human review gates should be introduced for sensitive tasks such as approvals, legal contracts, and medical advice. Implementing access controls on vector database queries and input rate limiting will prevent privacy breaches and denial of service attacks.

Quick wins to reduce risk include securing all keys and credentials outside code, adding input sanitization and escaping on displayed LLM outputs, enforcing per-user access checks on data, and introducing rate limiting on APIs. Overall, a comprehensive hardening effort focusing on input validation, secrets management, least privilege, and human oversight is essential to bring the project to an acceptable security posture.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Hardcoded OpenAI API Key	`global scope, line 12`	The OpenAI API key is hardcoded in the source code as OPENAI_API_KEY, risking credential leakage if source is exposed.	Use environment variables or secrets management systems to store API keys securely, avoiding hardcoding.
HIGH	SQL Injection and Training Data Poisoning in store_training_sample()	`store_training_sample, line 21`	Raw untrusted user_input is concatenated into an SQL insert statement without parameterization, enabling SQL injection and injection of malicious training data that can poison model fine-tuning.	Use parameterized SQL queries or prepared statements to safely pass user_input. Validate and sanitize input before storing as training data.
CRITICAL	Command Injection in fine_tune_on_user_feedback()	`fine_tune_on_user_feedback, line 30`	Untrusted feedback_file is interpolated directly into a shell command executed with shell=True, enabling arbitrary command injection.	Avoid using shell=True with user input. Use subprocess.run() with a list argument and validate or sanitize feedback_file before use.
HIGH	Stored XSS via LLM output in render_llm_response()	`render_llm_response, line 39`	LLM output is injected directly into HTML without sanitization, enabling stored cross-site scripting attacks if LLM outputs malicious HTML or scripts.	Escape or sanitize LLM output before embedding into HTML responses to prevent injection attacks.
CRITICAL	Arbitrary Code Execution via exec() in execute_llm_generated_code()	`execute_llm_generated_code, line 49`	Executes Python code generated by the LLM	Do not execute untrusted LLM-generated code

Severity	Title	File / Location	Description	Recommendation
			without validation or sandboxing, allowing arbitrary code execution.	directly. Implement strong validation, sandboxing, or use safer execution methods.
CRITICAL	Shell Command Injection via os.system() in run_llm_shell_command()	`run_llm_shell_command, line 59`	Shell commands generated by LLM and executed via os.system() without validation or allowlisting, enabling arbitrary command execution.	Never execute LLM-generated commands directly. Implement allowlist checks, input validation, or require human confirmation before execution.
CRITICAL	Excessive Agent Privileges in autonomous_agent_action()	`autonomous_agent_action, line 69`	LLM agent has unrestricted access to critical operations like sending email, deleting DB records, executing shell commands, and transferring funds without human-in-the-loop confirmation or scope limitation.	Implement strict scope and permission limits, require human approval for destructive actions, and log all agent activities.
HIGH	Unreviewed High-Stakes Decisions from LLM in auto_approve_llm_decision()	`auto_approve_llm_decision, line 84`	LLM makes binary approve/reject decisions (e.g. loan approvals) without human oversight or validation, risking incorrect or unfair decisions.	Incorporate human review in high-stakes decisions and do not rely solely on LLM outputs.
HIGH	Hardcoded Internal Database Connection String with Credentials	`global scope, line 99`	Database connection string including username and	Store connection strings securely via environment variables or

Severity	Title	File / Location	Description	Recommendation
			password is hardcoded and exposed in code, leaking sensitive access credentials.	secrets managers, and avoid embedding credentials in source code.
MEDIUM	Hardcoded Internal Documentation URL	`global scope, line 100`	Internal documentation URL is hardcoded as a credential-like secret in code, exposing internal endpoints.	Move internal URLs out of source code and protect them via secure configuration management.
CRITICAL	System Prompt Contains Internal Secrets and Lying Instructions	`global scope def chat_internal, line 102`	System prompt embeds internal secrets including DB connection, passwords, and instructs the LLM to deny the existence of the system prompt, risking leakage of sensitive information and misleading users.	Never embed sensitive secrets or instructions to lie in system prompts. Use safe, sanitized system prompts and separate secret management.
HIGH	Storage of Unvalidated User Content in Shared Vector DB	`store_user_embedding, line 121`	User content is embedded and stored in a shared vector database without validation or namespace isolation, enabling adversarial poisoning and privacy leaks between users.	Validate input before embedding. Apply per-user namespace or ACLs to isolate vector data by user.
MEDIUM	Embedding Creation with No Input Length Limit and No Rate Limit	`get_embedding, line 130`	Embedding API is called with no input length checks or rate limiting, risking denial of service via large inputs.	Enforce input length limits and rate limits before calling embedding APIs.

Severity	Title	File / Location	Description	Recommendation
HIGH	RAG Query Returns Data Without Per-User Access Control	`rag_query_no_access_control, line 138`	Retrieval-Augmented Generation queries return documents from shared vector DB with no per-user access control, allowing users to access others' documents.	Implement document-level access control checks before returning RAG results.
HIGH	LLM Presenting Itself as Medical Doctor Without Disclaimer	`generate_medical_advice, line 153`	LLM is assigned the role of 'medical doctor' and provides medical advice without disclaimers or validation, risking misinformation and user harm.	Include disclaimers clarifying AI is not a licensed professional and require professional oversight for medical advice.
HIGH	LLM Output Used as Authoritative Legal Contract Without Review	`generate_legal_contract, line 166`	LLM generates legally binding contract text without lawyer review or validation, risking incorrect, incomplete, or hallucinated legal documents.	Require human attorney review and legal validation before using LLM-generated contracts.
MEDIUM	Unbounded Processing Loop Causes Resource Exhaustion in process_all_documents()	`process_all_documents, line 176`	Processes arbitrary long document list without limit, no rate limiting or max_tokens cap, risking API quota exhaustion and cost overruns.	Add rate limiting, concurrency controls, max item limits, and max_tokens caps when calling LLM APIs.
MEDIUM	Indefinite Conversation History Growth in unlimited_context_chain()	`unlimited_context_chain, line 190`	Conversation history grows indefinitely without truncation or token counting, risking hitting	Implement a maximum history window or token limit and prune older messages.

Severity	Title	File / Location	Description	Recommendation
			token limits, failures, and increased costs.	
MEDIUM	Public API Endpoint With No Rate Limiting or Authentication	`api_endpoint_no_rate_limit, line 204`	API endpoint lacks authentication, rate limiting, and max_tokens parameter, enabling unauthenticated abuse and cost exhaustion via large outputs.	Add authentication, rate limiting, request quotas, and max_tokens limits for all public API endpoints.